Statistical and Machine Learning

Neural Networks for Images

Instructions

All questions should first be discussed in pairs or small groups. Afterwards, we shall discuss them together. Solutions will be distributed after each session.

Question 1

From MLPs to CNNs. Multilayer perceptrons (MLPs) face three fundamental problems when applied directly to image data.

(a) Briefly describe each of the three problems with using a plain MLP layer on image inputs $\mathbf{x} \in \mathbb{R}^{W \times H \times C}$.

(b) What is *translation invariance*, and why is it desirable for image recognition? Illustrate with an example of what goes wrong without it.

(c) How do Convolutional Neural Networks (CNNs) address these three problems?

Question 2

Convolutions and cross-correlation. The 2D cross-correlation of a filter $\mathbf{W}$ of size $H \times W$ with an image $\mathbf{X}$ is defined as:

$$[\mathbf{W} \circledast \mathbf{X}](i,j) = \sum_{u=0}^{H-1} \sum_{v=0}^{W-1} w_{u,v}\, x_{i+u,\, j+v}$$

(a) What is the difference between *convolution* and *cross-correlation*? Under what condition do the two operations coincide?

(b) Compute the output $\mathbf{Y}$ of convolving the following 3×3 image $\mathbf{X}$ with the 2×2 filter $\mathbf{W}$ (using no padding):

$$\mathbf{W} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \qquad \mathbf{X} = \begin{pmatrix} 2 & 1 & 3 \\ 0 & 4 & 1 \\ 1 & 2 & 0 \end{pmatrix}$$

(c) How does convolution implement *feature detection*, and what is a *feature map*?

Question 3

Padding, stride, and parameter efficiency. Consider a convolutional layer applied to an input of size $x_h \times x_w$, using a filter of size $f_h \times f_w$, with padding p_h, p_w on each side and stride s_h, s_w.

(a) Write down the formula for the output size of the convolutional layer.

(b) What is *valid convolution*? What is *same convolution*? What padding p (expressed in terms of filter size f) achieves same convolution?

(c) How does the Toeplitz-like matrix structure of the CNN weight matrix reduce the number of parameters compared to a dense MLP layer? Briefly explain the roles of *sparsity* and *weight tying*.

Question 4

Pooling and normalization. Pooling layers and normalization layers are both important building blocks of CNNs, but serve different purposes.

(a) Explain the difference between *max pooling* and *average pooling*. What property do pooling layers promote, and how does this differ from the *equivariance* property of convolutional layers?

(b) What is *global average pooling*, and what practical advantage does it offer?

(c) Batch normalization standardizes the activations within a minibatch $\mathcal{B}$. Write down the batch normalization transformation, define all terms, and explain the roles of the learnable parameters $\boldsymbol{\gamma}$ and $\boldsymbol{\beta}$.

(d) Why does batch normalization behave differently at *training time* versus *test time*, and how is this difference handled in practice?

Question 5

CNN architectures. Several landmark CNN architectures have been developed for image classification, each introducing key innovations.

(a) LeNet (1998) was an early CNN for digit recognition. What was its main application, and what accuracy did it achieve on MNIST compared to an MLP?

(b) AlexNet (2012) marked a turning point in deep learning. List **three** architectural or training innovations that distinguished it from LeNet.

(c) ResNet (2015) introduced *residual blocks* with skip connections. Write down the residual block update equation and explain, in words, why skip connections help train very deep networks.

Question 6

Beyond image classification. CNNs can be applied to a variety of discriminative vision tasks beyond single-label classification.

(a) Explain the difference between *image classification*, *image tagging*, and *object detection*. Why does image tagging replace the softmax output with independent logistic units?

(b) Distinguish between *semantic segmentation* and *instance segmentation*. Give a concrete example illustrating the difference.

(c) Describe the *encoder–decoder* architecture used for semantic segmentation.

Question 7

Generating images by inverting CNNs. A CNN trained for classification can be "inverted" to generate images.

(a) Explain how a discriminative classifier $p(y \mid \mathbf{x})$ can be converted into a (conditional) generative model $p(\mathbf{x} \mid y)$. Why is a prior $p(\mathbf{x})$ over images necessary?

(b) The Langevin-style update for image generation is:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \epsilon_1 \frac{\partial \log p(\mathbf{x}_t)}{\partial \mathbf{x}_t} + \epsilon_2 \frac{\partial \log p(y = c \mid \mathbf{x}_t)}{\partial \mathbf{x}_t} + \mathcal{N}(\mathbf{0}, \epsilon_3^2 \mathbf{I})$$

Explain the role of each of the three terms (ϵ_1, ϵ_2, ϵ_3). What happens when $\epsilon_3 = 0$?

(c) What is *neural style transfer*? Define the three components of its energy function $\mathcal{L}(\mathbf{x} \mid \mathbf{x}_s, \mathbf{x}_c)$ and explain what the *Gram matrix* captures and why it is used for the style loss.